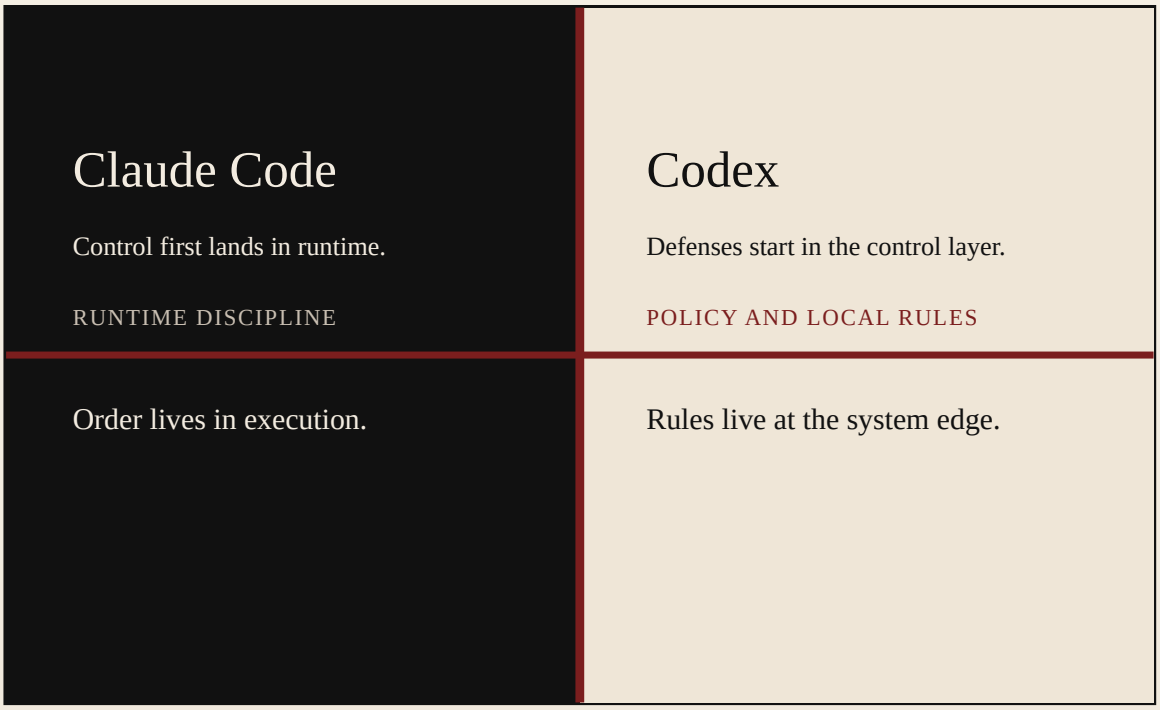

The Harness Design Philosophies of Claude Code and Codex

Convergence through different roads, or separate branches?

System builders are just people
who write inevitable damage
into control flow early enough
that it cannot govern them later.

CONTROL / LOOP / POLICY / STATE / LOCAL GOVERNANCE / VERIFICATION

A comparative engineering notebook on control layers, state governance, tool boundaries, and organizational discipline

agentway.dev

From using agents to shipping an agent PoC

https://agentway.dev

2026-04-01 · rev 37dfc2

Contents

Introduction

Subtitle: convergence, divergence, and the engineering choices behind two coding-agent systems.

This is the English edition of the comparative book. It is structured to mirror the Chinese edition while emphasizing the points that matter to readers making architectural choices in English-language teams.

What this collection offers

- A reading map that explains how this book sits beside the first volume.
- A series of chapters that trace the same harness concerns (control plane, continuity, tool policy, hooks, delegation, and verification) through both Claude Code and Codex.
- Appendices that map sources and boil the comparison down to practical checklists.

Use the navigation on the left to follow the chapter order above. If you just want the conclusion, start at Chapter 7.

Reading Map: How to Read Book One and This Comparative Book Together

If you treat both directories as "books about AI coding agents," the easiest misread is to assume they are duplicate documents. They are not. Their division of labor is clear.

Book One is single-system dissection. It uses Claude Code as a specimen to explain why a controllable agent must grow organs such as a control plane, query loop, tool permissions, context governance, recovery paths, multi-agent verification, and team institutions. Its central question is: what internal skeleton does a harness need in order to work continuously in real engineering environments?

This comparison book is comparative dissection. It no longer asks only why Claude Code is designed this way, but puts Claude Code and Codex side by side to compare how each admits model unreliability and places order at different layers. Its central question is: when both systems are building a harness, which choices are consensus and which are just different engineering routes?

There is now one more use for that comparison: once you carry these judgments into third-party harnesses, it becomes easier to spot a common failure mode. Many systems also advertise memory, skills, compaction, and multi-agent support, yet their context-governance axis still amounts to pushing more text into prompt first and truncating or rescuing later. That route looks like "more information," but in practice it often burns more tokens while weakening working semantics.

In other words, you can read them as two sequential steps in one research program:

- Step 1: extract general principles of Harness Engineering in Book One.
- Step 2: observe how those principles land differently across two concrete systems in this comparison book.

Start with Book One: Nine Structural Judgments from Claude Code

Book One has a stable throughline and can be compressed into nine judgments.

1. The first duty of a harness is to stop the model from damaging engineering environments; capability amplification usually sits on top of that.
2. In an agent system, prompt is better understood as part of the control plane, while still carrying some material that older systems would have treated as persona copy.
3. The query loop is the heartbeat of an agent; the core question is how one turn connects to the next.
4. Tools are more than a capability list; they are execution interfaces constrained by approval, orchestration, and interrupt semantics.
5. More context is not always better; memory, `CLAUDE.md`, and compact are budget-governance mechanisms.
6. Errors should not be treated as edge material; recovery paths work better when designed as first-class paths.
7. Multi-agent value mainly comes from role partitioning and independent verification; the appearance of duplication is secondary.
8. Team rollout cannot depend on individual tricks; rules must crystallize into reusable institutions.
9. These insights can eventually become a reasonably stable Harness Engineering principle checklist.

If you read only Book One, you get a strong impression: Claude Code's systemic character is runtime governance. It first cares about:

- how the session keeps running,
- how tools avoid trouble,
- how recovery avoids dead loops,
- and how verification avoids ritualism.

Then Read This Comparison Book: Same Problem, Different Starting Points

Based on that foundation, this book splits comparison into explicit layers.

Control Plane

Claude Code behaves more like dynamic prompt assembly. A lot of order is packed into runtime prompt composition, session state, and context governance.

Codex behaves more like explicit control-layer engineering. It modularizes and types instruction fragments, approval policies, tool schema, thread, rollout, and hooks as much as possible.

Continuity

Claude Code places continuity in the main loop and emphasizes query-loop heartbeat discipline.

Codex distributes continuity across thread, rollout, and state bridge, emphasizing structured state ownership and recovery.

Tools and Permissions

Claude Code leans toward runtime constraints: approval at call time, interrupts, and preventing risky actions from landing directly.

Codex leans toward policy language and tool contracts: schema, approval policy, sandbox, and exec policy as explicit organs.

Local Governance

Claude Code tends to absorb local experience into field memory: `CLAUDE.md`, memory, skills, and workflow constraints.

Codex tends to mount local institutions onto structured injection and event systems: instructions, skills, hooks, and explicit tool boundaries.

Multi-Agent and Verification

Claude Code emphasizes multi-agent role partitioning at runtime, and insists verification must be independent from implementation.

Codex emphasizes explicit delegation, persistent state, and tool-mediated collaboration so verification becomes a trackable capability, rather than remaining a polite final gesture.

What Becomes Clear When You Read Them Together

Putting Book One and this comparative volume together yields three fuller conclusions.

First, the comparison is centered mainly on the harness

Both books point to the same fact: the core challenge of AI coding systems is to keep models from losing control in terminals, filesystems, permission boundaries, and team institutions. Gains in eloquence matter, but they sit downstream of that problem.

Second, the main difference between Claude Code and Codex is where order is placed

Claude Code is closer to a system grown from runtime incident pressure, prioritizing continuity, recovery, and field governance.

Codex is closer to a system grown from explicit structural design, prioritizing control-layer naming, policy expression, boundary clarity, and composability.

Third, followers usually gain more by identifying their own dominant uncertainty

If your biggest pain is long-session instability, brittle recovery, and skipped verification, start with the runtime discipline emphasized in Book One.

If your biggest pain is scattered rules, fuzzy permission boundaries, unstable tool contracts, and non-reproducible team behavior, start with the explicit-control-layer lessons this comparison extracts from Codex.

And if you encounter a system whose continuity depends mainly on stacking bootstrap text, identity files, skill catalogs, and workspace prose into prompt, do not be too impressed by the apparent fullness of context. In many cases, that points to an intermediate state that has not yet truly governed context, rather than a mature third road.

Recommended Reading Order

If you are new to this material set, use this order:

1. Read the Preface first to confirm that this material is comparing where order is located, not merely listing features.
2. Read Chapter 1 Why Put Claude Code and Codex Side by Side to establish problem framing.
3. Then read Chapters 2 through 6 in sequence across five axes: control plane, continuity, tool governance, local institutions, and multi-agent verification.
4. If you only want a quick synthesis, jump to Chapter 7 Converging Destinations, Diverging Branches.
5. If your goal is to build systems yourself, finish with Chapter 8 If You Build One Yourself: Who to Learn from, and What to Learn First. That chapter now also includes a three-path diagram showing why some harnesses still feel expensive and disorderly even when their context looks full.

One-Sentence Summary

Book One explains why a controllable agent must grow this way.

This comparative book explains why two serious harness systems still grow differently.

Preface: Two harnesses, not one accessory pack

Comparing two AI coding harnesses is easy to get wrong if you only line up feature checkboxes. Both Claude Code and Codex have skills, sandboxes, and sub-agents. But seeing shared terminology does not mean their skeletons are the same. It is like noting both cities have bridges—the real question is which river they are trying to cross.

This book aims to compare the bones, not the labels.

Both systems admit something that marketing often sweeps under the rug: a model cannot be trusted to execute without constraints. Shells, filesystems, permissions, tool calls, teams, long sessions, and recovery paths are messy realities. A harness is that messy reality; it is the apparatus that keeps an unreliable model from burning the environment down. When that harness exists, you no longer measure success by how eloquently the model speaks—you measure it by where and how the system places guardrails.

In the first volume I treated Claude Code as a specimen and extracted the general principles of Harness Engineering. That was single-system anatomy. Here the goal is to place Claude Code beside another serious harness with a different lineage. Only in comparison do many design choices reveal themselves as one engineering path among many.

Codex is worth comparing precisely because it is not a clone. A glance at its `core/src/lib.rs` shows a deliberate program: threads, rollouts, state bridges, instructions, skills, hooks, sandboxing, exec policy, tools—each one composed into an explicit module. The ambition is to make the control layer composable, serializable, and policy-aware instead of hiding it inside a bowl of runtime intuition.

Claude Code, by contrast, feels like something grown under the pressure of

its runtime loop. Look at `src/query.ts` and the surrounding compacting, tool orchestration, permission handling, interrupts, and recovery logic. Most of its magic answers the question "how does this round hand off safely to the next round?" The harness first has to keep running continuously; once continuity is stable, then one can polish the rules between phases.

Both systems agree that models are unreliable. That shared conviction matters. Once you admit the model cannot be left inhabiting shell, files, permissions, and long conversations on its own, a harness must grow—prompt stacking, state persistence, approval, context governance, recovery flows, verification, and local conventions. Only the placement of those organs differs.

This book does not reproduce Claude Code or Codex source, and it does not transcribe long implementation excerpts. That boundary is not mysterious: respect for proprietary code, as well as a desire to keep the argument in the engineering analysis. What follows is a comparison of how these systems think about themselves, not a copy-paste of their implementations.

If I had to summarize in one sentence:

> Claude Code and Codex are aligned not because they both call tools, but because neither is willing to treat the model as a free-moving executor.

As for their differences, they are more interesting.

Claude Code approaches harnessing through runtime discipline. It worries about session rupture, tool chaining, compact behavior, user interjections, and what happens if a forked agent dies mid-conversation. It has the feel of someone who has cleaned up after messy shifts in a real operations center—preferring to solve "smart" problems through craftsmanship in control and recovery flows.

Codex approaches it through structured control. Instruction fragments, threads, approval policies, tool schemas, hook events, and exec policies are all spelled out. It values naming, explicit boundaries, and configuration. Codex does not rely on tacit understanding; it prefers to declare marker types and inject them deterministically.

Both routes are valid. They are just valid in different ways.

This is not an article about winners and losers. Valuable engineering comparison helps identify path dependencies, not rank execution speed. The way

you choose to constrain an unreliable model determines what your harness becomes. Where your team places control is where the system will grow its flesh. Instead of asking "what is the best practice?," ask "what uncertainty am I countering, and where am I willing to anchor order?"

The next seven chapters start from that question.

@wquguru
2026.04.01
Claude Code fools-day source leak

P.S. Read the online version at harness-books.agentway.dev/book2-comparing for a richer experience.

Chapter 1: Why we compare Claude Code and Codex

1.1 Because both distrust the model

Calling Claude Code and Codex "models that write code" misses the point. The interesting comparison is not "who has more buttons," it is how each harness admits a fact marketing often disguises: the model cannot be trusted to operate unbounded shell, files, network, or state. It hallucinates, forgets context, and imagines confidence beyond correctness. Once that model touches a terminal, a filesystem, or a multi-round session, the issue is no longer "was the answer right?" but "who cleans up the mess?"

Claude Code looks like a system born from incident reviews. Read `query.ts`, `toolOrchestration.ts`, `compact.ts`, and the surrounding prompts—you see a system that imagines failure, fatigue, and rollback. Codex is equally honest, but it expresses distrust through explicit modules: threads, rollouts, fragment instructions, exec policies, sandboxing, and tool schemas. Each declares its responsibilities instead of leaving them to the model's intuition.

So this comparison is about how each harness domesticates unreliability—not about who can hit more commands.

1.2 Claude Code: runtime-first harnessing

Claude Code's strengths cluster around the main loop because it starts from one question: "The model is already cycling. How do we make sure one round doesn't sabotage the next?"The system is less interested in "pretty prompts"and more interested in prompt layering, compacting, permission gating, interrupts,

tool orchestration, and forked agent life cycles. The work happens at runtime: managing message inflation, tool output rerouting, context trimming, child-agent isolation, and independent verification phases. This harness grows out of real shifts—so it solves "dirty work" through control flow, not just through elegant interfaces.

1.3 Codex: structured control from the start

Codex takes a different path. Its design treats instructions as typed, bounded fragments. `AGENTS.md`, `skills`, and user fragments appear in the prompts as marked sections with start and end tokens. Tools are schema-first objects. Exec policy is a distinct crate with policies, rules, evaluations, and parsers. The emphasis is on making order explicit, composable, and explainable.

This yields two immediate outcomes: the control plane is easier to explain (you can trace why a fragment is present), and it is easier to programmatically evolve (add new visibility, merge, or precedence rules without rewriting the runtime).

1.4 Same question, different starting points

Both systems try to keep an unreliable model from doing unacceptable things. Claude Code begins by asking how a running loop can stay coherent; Codex begins by asking how control information can be turned into explicit, composable structures. Claude Code gravitates toward runtime discipline; Codex gravitates toward typed infrastructure.

1.5 Why the difference matters

The direction a team takes often determines how its culture evolves. Emphasize runtime continuity, and you will obsess over recovery, interruption, state pollution, tool choreography, and long-session reliability. Emphasize explicit control, and you will obsess over instruction boundaries, config hierarchies, tool schemas, policy languages, and persistent thread state.

Both paths are valid. The mistake is mixing them without clarity and losing both runtime discipline and institutional order.

1.6 The take-away

Claude Code and Codex are harness design philosophies, not feature checklists. They converge on the acceptance that models are untrustworthy; they diverge in what they build around that admission. This book unpacks that divergence.

Chapter 2: Two control planes—dynamic prompt layers vs structured fragments

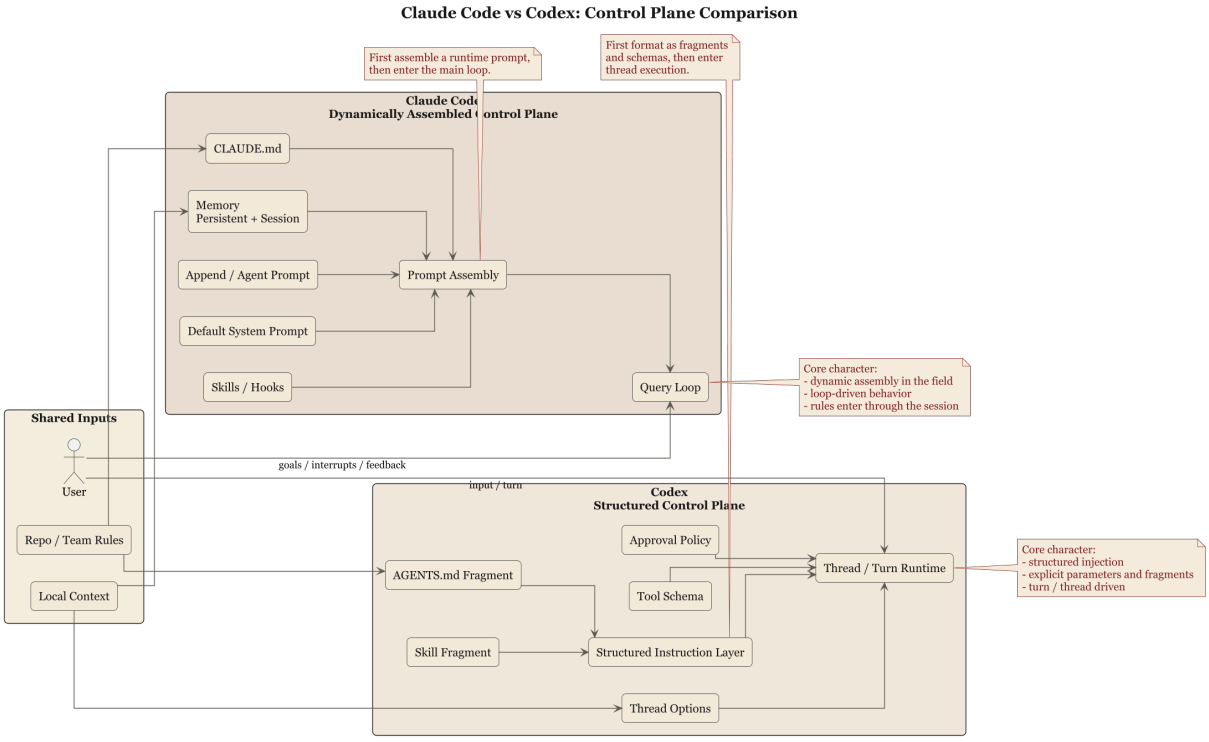

Figure 1: Control plane comparison diagram

2.1 Control is not about tone

Talking about prompts as if it were just a tone exercise is misleading. Claude Code and Codex both treat prompts as parts of a behavioral control plane. The difference lies not in the copy but in the mechanisms that assemble and surface it.

Claude Code assembles prompts dynamically. Baseline text, append prompts, agent roles, `CLAUDE.md`, memory, and output styles are layered at runtime according to the task, tools, and team context. The art is in priority, conflict resolution, and how each layer folds into the next. The control plane is a living pipeline that must adapt every loop.

Codex treats instructions as structured fragments. `AGENTS.md`, user messages, and skills become chunks with clear markers, start and end boundaries, and serialization rules. Tools and instructions stop being free-form narrative and become identifiable contextual units. This gives the control plane traceability and the ability to grow more governance artifacts without losing clarity.

2.2 Claude Code's busy assembly line

Claude Code's system prompts are not fixed documents; they are a production line. Defaults provide the foundation; append prompts drop requirements; agent prompts add roles; `CLAUDE.md` and memory inject local conditions. This flexibility lets the same loop handle different scenarios, but it makes the order of assembly critical: wrong ordering can dilute instructions or let conflicts slip through.

Because of this, runtime governance is essential. Control is constantly injected, overwritten, compressed, or pruned when tasks shift. The query loop recalculates "what matters now" each round. Claude Code's guiding intuition is: control has to follow the scene—it cannot be frozen into static rules.

2.3 Codex's filing-room approach

Codex insists on identifiable fragments. Names like `ContextualUserFragmentDefinition` highlight type, boundaries, wrapping rules, and transformation into messages. AGENTS, skills, and user instructions are not just textual—they are tagged contextual units that the system can recognize and manipulate. This yields stronger debuggability and a path to more programmatic governance, because every instruction already fits into a type hierarchy.

And this is not merely a matter of elegant naming. In `fragment.rs`, Codex really defines constants such as `AGENTS_MD_START_MARKER`, `AGENTS_MD_END_MARKER`,

`SKILL_OPEN_TAG`, and `SKILL_CLOSE_TAG`, then uses `ContextualUserFragmentDefinition::wrap()` and `into_message()` to turn those fragments into `ResponseItem::Message`. In `user_instructions.rs`, `UserInstructions` serializes the directory into the line `# AGENTS.md instructions for ...`, while `SkillInstructions` carries explicit `<name>` and `<path>` fields. In other words, Codex tries hard not to make the model guess where a rule came from.

2.4 CLAUDE.md vs AGENTS.md

The distinction between CLAUDE.md and AGENTS.md is revealing. Claude Code's CLAUDE.md feels like a local bulletin board—an on-site rule set tailored to a directory or workspace. It is pragmatic, local, and mission-focused. Codex's AGENTS.md, by contrast, names scope, priority, and inheritance. Even when `child_agents_md` is not present, Codex injects scoped instructions to clarify applicability. Claude Code brings local rules into the conversation; Codex brings local rules into the institution.

2.5 The trade-offs

Claude Code's runtime assembly is flexible but hard to formalize. Fragmented instruction is explicit but structural. Claude Code builds experience-driven control; Codex builds institutional control. The former trades explicitness for agility; the latter trades simplicity for clarity and maintenance cost.

2.6 This chapter's conclusion

Claude Code views prompts as dynamic runtime builds; Codex views instructions as identifiable fragments. One feels like a production floor; the other feels like a bureaucracy. The right choice depends on whether your primary worry is volatile sessions or unclear rule sources.

Chapter 3 Where the Heartbeat Lives: Query Loop Versus Thread, Rollout, and State

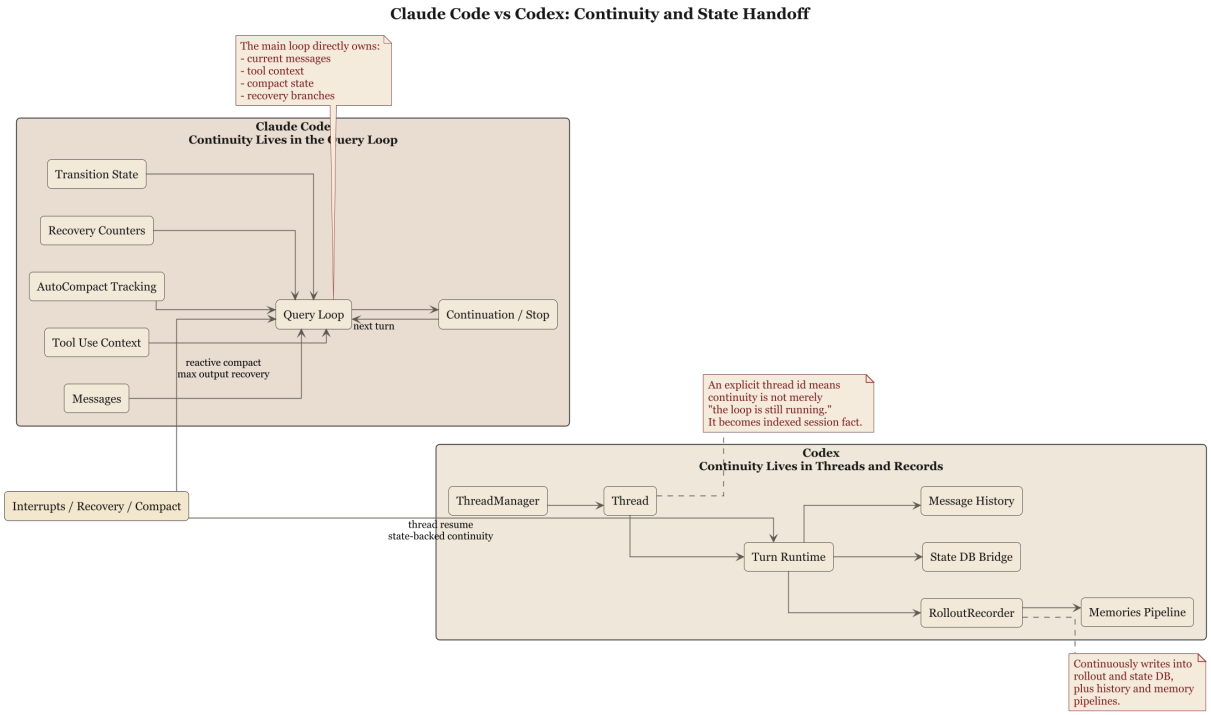

Figure 2: Continuity Comparison

3.1 The core of an agent system is continuity

If you understand an agent system as “multi-turn chat,” that is like understanding a database as “a patient notebook.” It is not entirely false, but it hides the real architectural problem.

The hard problem in an agent system is continuity:

- what happened last turn, and how this turn picks it up
- how tool results are merged back into the session
- how interruption gets closed out
- how overgrown context is reorganized
- whether failure should trigger retry, compaction, or faithful reporting

These questions determine whether a system is really an agent, rather than merely a question-answering interface that happens to support tool calls.

The difference between Claude Code and Codex here is more revealing than almost any feature table.

3.2 Claude Code: continuity is compressed into the main loop

Claude Code's axis is easy to locate around `query()` and `queryLoop()`. It pushes many crucial issues into loop state:

- current message sequence
- tool-use context
- compact tracking
- output-token recovery counters
- pending summaries
- turn counts
- transitions

That means Claude Code answers the question "how does an agent stay alive?" in runtime terms. Continuity is maintained mainly by the loop. The skeleton therefore feels more like a conversation engine that keeps correcting itself than a system driven first by a strong external state model.

Its advantage is very concrete. Much of the real trouble in sessions happens exactly inside the loop: tool-return ordering, abrupt truncation of model output, prompt-too-long events, history snips, microcompact, and user interjections. Claude Code does not avoid these issues. It treats them as legitimate states inside the loop.

That gives the design a rough engineering texture. It is not always elegant, but it is often more robust.

3.3 Codex: continuity is split across thread, rollout, and state bridges

Codex looks more ledger-like. Even from `core/src/lib.rs`, continuity is clearly not concentrated in one large loop. It is distributed across:

- `codex_thread`
- `thread_manager`
- `rollout`
- `state_db_bridge`
- `state`
- `message_history`

Then look at `sdk/typescript/src/thread.ts`. There, `Thread` is already a first-class concept that outside developers can understand and manipulate directly. A thread has an `id`, can `runStreamed()` or `run()`, and `thread.started` reports back the thread ID. Turn-level execution conditions such as approval policy, working directory, sandbox mode, network access, and additional directories are all exposed as explicit parameters tightly coupled to thread execution.

You can see a very literal kind of thread sovereignty there. `runStreamedInternal()` does not just hand input to the backend. It first calls `normalizeInput()`, separating prompt text from local images, then creates an output-schema file with `createOutputSchemaFile()`. The actual `_exec.run()` call carries `threadId`, `approvalPolicy`, `sandboxMode`, `workingDirectory`, `networkAccessEnabled`, and `additionalDirectories` as explicit execution parameters. When the event stream emits `thread.started`, the thread object updates its own `_id`. That means thread is not outer packaging. It is a genuine carrier of turn semantics.

What matters most is that continuity is no longer merely "the loop is still going." It becomes "a thread is being recorded and constrained by a more explicit state structure." The existence of rollout is especially telling: Codex cares deeply about replayability, indexing, persistence, and visibility outside the immediate live session.

That makes the system feel more like a true execution recorder and less like a live conversation manager alone.

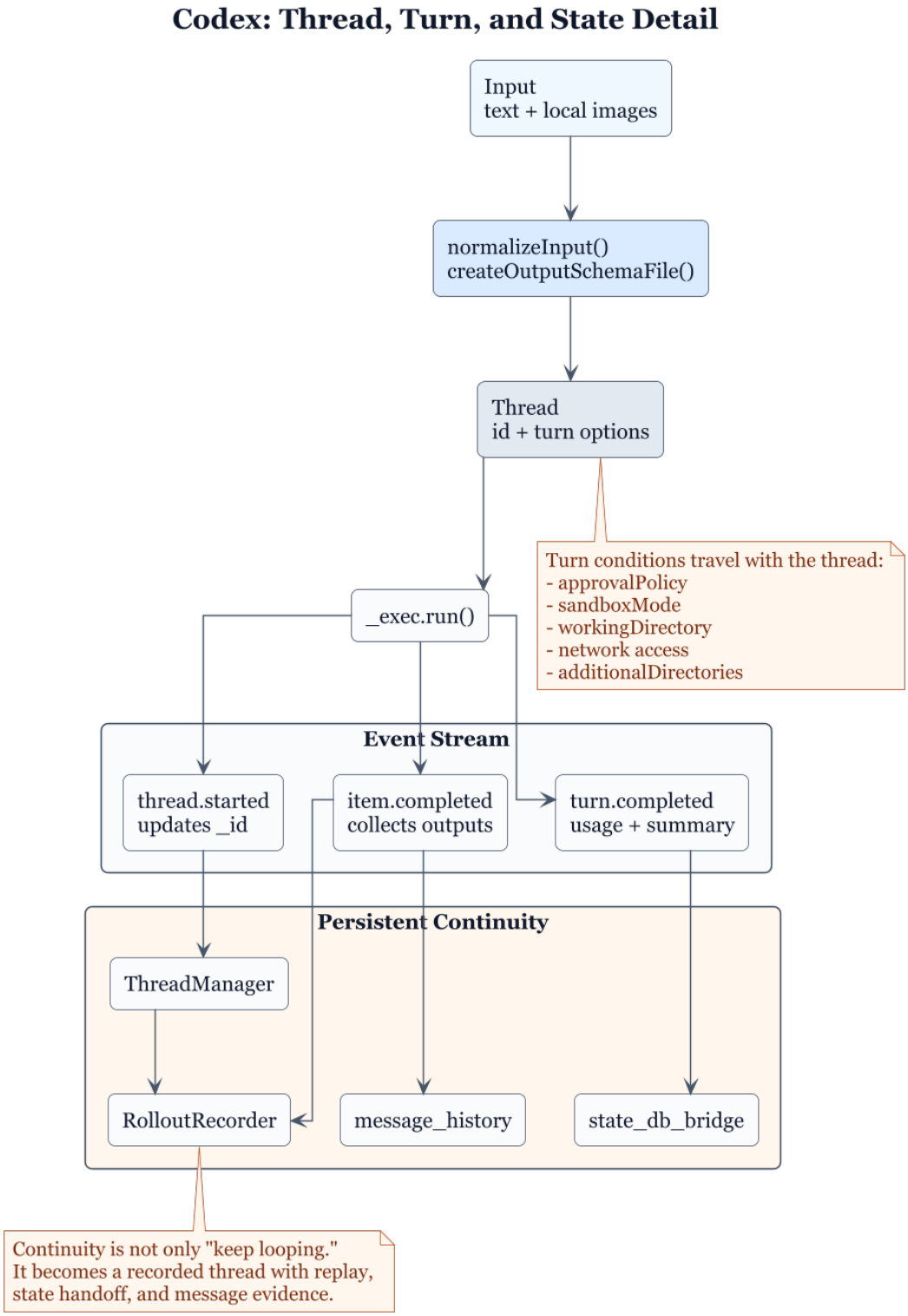

Figure 3: Codex thread-turn-state detail

3.4 The difference is where state sovereignty lives

One clarification matters. Claude Code obviously has state. Codex obviously has loops. The difference is not presence or absence, but sovereignty.

Claude Code gives more sovereignty to the query loop. It treats "how the session continues" as a runtime-core problem, and many issues are meant to be solved directly inside the loop.

Codex gives sovereignty more explicitly to thread and rollout structures. It treats continuity as something that should not remain a by-product of internal control flow; it should become an explicit fact carried by thread and state infrastructure.

That is why `Thread` in the TypeScript SDK already feels like a product concept rather than an implementation detail. Developers are allowed to think about agent turns directly through the thread abstraction.

Claude Code's query loop is more like the engine room. You know it matters, but not every user has to organize their mental model explicitly around it.

3.5 Different implications for recovery and auditability

This placement of state directly affects recovery and auditability.

Claude Code's recovery strength comes from proximity to the scene. Many problems are discovered and handled inside the loop itself: reactive compaction, output-token recovery, and tool-interruption cleanup. It does not first need to lift the problem into a higher-order state model before deciding what rollback should mean.

Codex's recovery strength is more likely to appear in traceability. Threads have IDs, rollouts have records, and state bridges plus message history provide clearer external structure. That makes it easier for the system to answer the question, "What exactly happened last turn?" rather than relying on runtime recollection alone.

If you read `core/src/lib.rs` together with that SDK layer, the archive-minded design becomes even clearer. Codex does not merely export `CodexThread`; it

also exposes modules and types such as `ThreadManager`, `RolloutRecorder`, `state_db_bridge`, and `message_history`. A system that places those organs near the root module is effectively admitting that continuity is not a side effect of looping. It is infrastructure.

In plain language:

- Claude Code is closer to an emergency crew on site
- Codex is closer to a dispatch center with archives

Both matter. The former is better at keeping execution going. The latter is better at explaining how that continuity is maintained.

3.6 Different effects on product and team interfaces

This difference also changes how teams integrate with the system.

If sovereignty lives in the loop, teams naturally organize around runtime questions:

- which failures need recovery
- which actions should be interruptible
- when compaction should trigger
- how tool results return to the main conversation

If sovereignty lives in threads and state structures, teams organize more around interface and governance questions:

- what the thread lifecycle is
- which events rollout should retain
- where the state store lives
- how approval policy becomes a turn-level option

So Claude Code is closer to making the agent operational first and embedding institutions afterward. Codex is closer to defining institutional interfaces first and letting the agent operate inside them.

3.7 Chapter conclusion

The conclusion can be stated plainly:

> Claude Code places more of continuity inside the query loop, while Codex places more of continuity inside thread, rollout, and state infrastructure.

The former emphasizes runtime heartbeat.

The latter emphasizes a persisted session substrate.

This is not an aesthetic difference. It is a distribution of system power. Whoever owns continuity defines the center of the harness.

The next chapter moves into the hardest layer of all: tools, sandboxes, approvals, and execution policy. Once shell enters the picture, romantic narratives usually leave on their own.

Chapter 4 Tools, Sandboxes, and Policy Languages: Who Stops the Model from Acting Too Fast

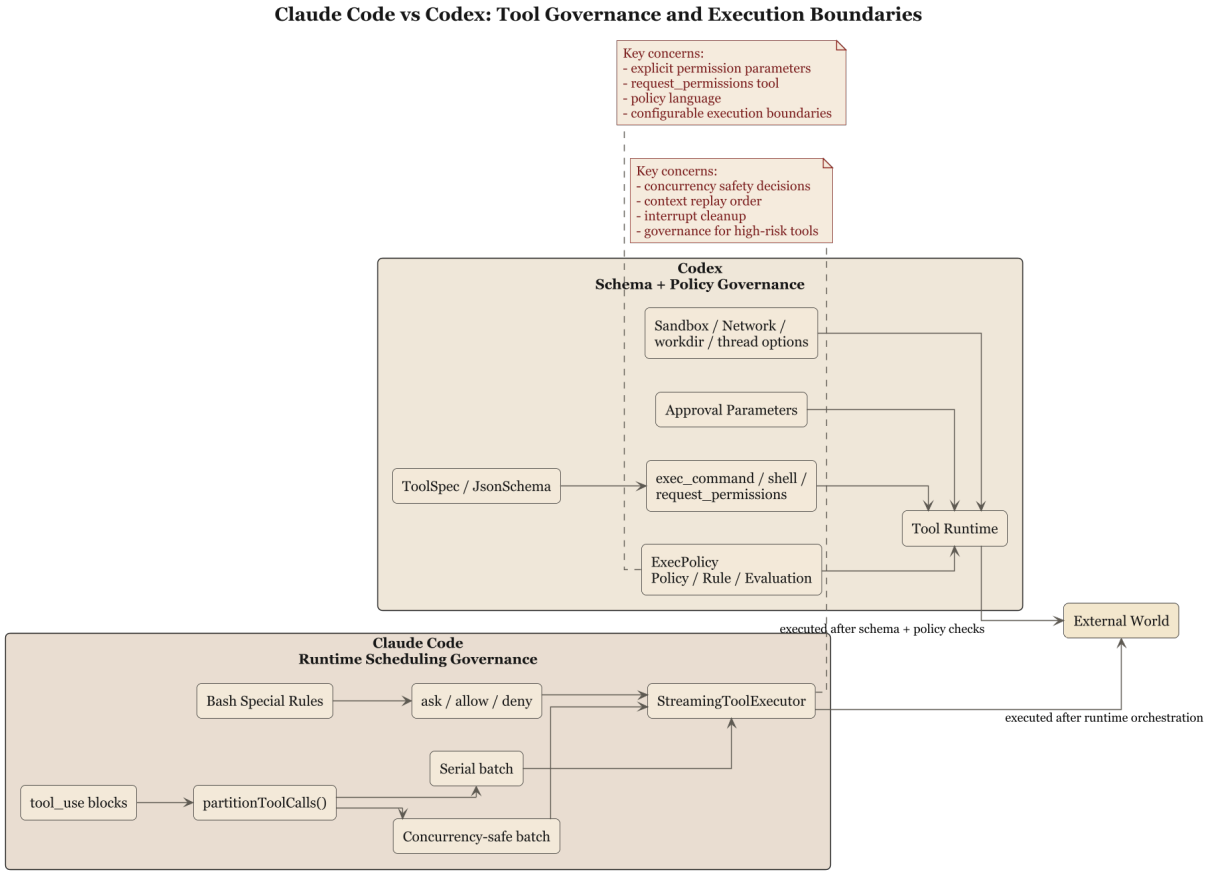

Figure 4: Tool Governance Comparison

4.1 The truly dangerous moment is the start of execution

When a model says the wrong thing, the usual cost is wasted time. When it runs the wrong command, it can take the directory, repository, processes, and workflow down with it. So what truly distinguishes AI coding systems is who owns the final interpretive authority before a tool starts acting.

Both Claude Code and Codex take this seriously, but they take it seriously in different ways.

Claude Code is closer to pulling tools into runtime scheduling discipline. It has `toolOrchestration.ts`, `toolExecution.ts`, `StreamingToolExecutor.ts`, `useCanUseTool.tsx`, Bash-specific prompts, and explicit `allow / deny / ask` semantics. It cares about whether this tool call may run, how it runs, whether it can run concurrently, whether the user has rejected it, how interruption works midway through execution, and how the result returns to context.

Codex instead starts by turning tools into typed interfaces. `tools/src/lib.rs` exports a whole set of tool constructors, while `local_tool.rs` defines schema-based structures for things like `exec_command`, `shell`, `shell_command`, and `request_permissions`. In Codex, tools are first a normalized API surface and only secondarily an execution unit.

4.2 Claude Code: runtime orchestration and constraints on dangerous action

Claude Code's tool system has a strong feel of field dispatch. Concurrency depends on schema and `isConcurrencySafe()`. Context modifications have to preserve replay order. Streaming tool execution must consider interruptions, synthetic results, and UI feedback.

What feels most like real engineering is that the system admits tool invocation is a process with consequences, not a single point action. In that sense, Claude Code's harness resembles a site supervisor attached to the model. Workers may work, but the supervisor must keep watching:

- which tool goes first

- which calls can be parallelized
- which must remain serial
- how results are accounted for
- what to do if work is halted halfway

Bash, especially, is treated with a near-obsessive explicitness. That may sound fussy, but mature systems are usually fussy around the most dangerous interface. Anyone who still approaches shell with youthful swagger probably has not accumulated enough incident memory yet.

4.3 Codex: tool schemas, approval parameters, and a policy engine

Codex is closer to expressing control over risky actions as formal interface constraints.

Take `local_tool.rs`. A tool such as `exec_command` explicitly owns fields like:

- `cmd`
- `workdir`
- `shell`
- `tty`
- `yield_time_ms`
- `max_output_tokens`
- `login`
- approval-related parameters

And `shell` / `shell_command` descriptions explicitly require `workdir`, warning against careless `cd` habits. This reveals an important design belief: Codex does not settle for "the runtime should quietly do the right thing." It wants correct usage patterns encoded in the tool definition itself.

More than that, approvals and escalation are modeled as explicit parameters, `request_permissions` exists as a separate tool, and `execpolicy` is implemented as its own crate. That language already goes beyond simple permission gates into an actual policy layer:

- `Policy`
- `Rule`
- `Evaluation`
- `Decision`
- parser

These names are practically announcing that execution boundaries have become a small policy language rather than a handful of `if / else` checks.

And that policy language is not rhetorical. In `local_tool.rs`, tool schemas explicitly mark required fields and disable stray properties with `additional_properties = false`, which narrows the room for the model to invent parameters. The descriptions for `shell` and `shell_command` even spell out that `workdir` should be set and `cd` should be avoided unless necessary. Meanwhile `execpolicy/src/lib.rs` exports not just a parser and `PolicyParser`, but also helpers such as `blocking_append_allow_prefix_rule` and `blocking_append_network_rule`. Codex is prepared not only to evaluate policy, but to amend it in structured ways.

4.4 Runtime approvals versus policy language

The split between Claude Code and Codex around tool risk can be compressed like this:

- Claude Code leans toward a runtime approval chain
- Codex leans toward explicit policy language and parameterized approvals

Claude Code's `ask / allow / deny` logic is tightly coupled to the scene of the tool call. The system can decide according to current context, tool type, user action, and session state. Its strength is sensitivity; its weakness is that rules can remain buried inside runtime logic.

Codex's exec-policy approach tries to pull rules outward and make them separately parsable and separately evaluable. The strength is better readability and portability of rules, and a much more natural fit for team-level governance. The cost is weight: the system feels heavier, and policy design must be treated as real design work rather than as commentary.

Put bluntly:

Claude Code is closer to a shift manager making the call on site.

Codex is closer to a company that writes the policy first and then checks whether the action is compliant.

4.5 Sandbox and approval are not merely security features

Many teams treat sandboxing, approvals, and permissions as security accessories. That is too light. In a coding agent, these things define what the product actually is.

If the system lets the model run arbitrary commands directly in a user directory, it is not simply "more powerful." It is an agent that transfers risk to the user. Conversely, if a system can explicitly express sandbox mode, network access, approval policy, additional directories, state-store location, and MCP tool approvals, then it is offering not just capability but behavioral boundaries.

Codex exposes these turn-level conditions in `thread.ts`, which shows it treats them as part of thread semantics rather than as hidden implementation detail. Claude Code instead pushes the boundary into runtime execution through tool handling, interrupts, permission hooks, and Bash restrictions.

That means the two systems differ at the level of product philosophy too:

- Claude Code is closer to "execute while being watched"
- Codex is closer to "declare the execution contract before starting"

4.6 MCP, external tools, and boundary migration

Both systems can attach more capabilities, but their differences remain.

Claude Code is closer to building a situational governance chain out of skill, hook, permission, and tool prompt. Its strength is getting local rules into the main loop together with the task.

Codex is more willing to pull external capabilities into one unified tool system. MCP resources, dynamic tools, and tool discovery interfaces in

`tools/src/lib.rs` show that extensions are expected to become schema-defined, rule-governed tools rather than temporary runtime understandings.

That is a consequential divergence. Once the ecosystem grows, the whole system becomes more dependent on how extensions obey the general rules. The team that thinks through boundary migration early is the team whose extension ecosystem is less likely to degenerate into a junk closet later.

4.7 Chapter conclusion

This chapter can be reduced to a hard sentence:

> Claude Code's tool governance depends more on runtime orchestration and situational approvals, while Codex's tool governance depends more on schemas, parameterized permissions, and an independent policy system.

The former is like an experienced foreman.

The latter is like a contractor with a compliance office and legal department.

If you look only at the fact that both can run commands, you miss the meaningful difference. The real question is who owns order before the tool starts moving.

The next chapter looks at a more grounded layer: skills, hooks, local rule files, and team institutions. Once a technical system has to enter a team, it eventually has to learn village law.

Chapter 5 Skills, Hooks, and Local Rules: How a System Learns to Respect Village Law

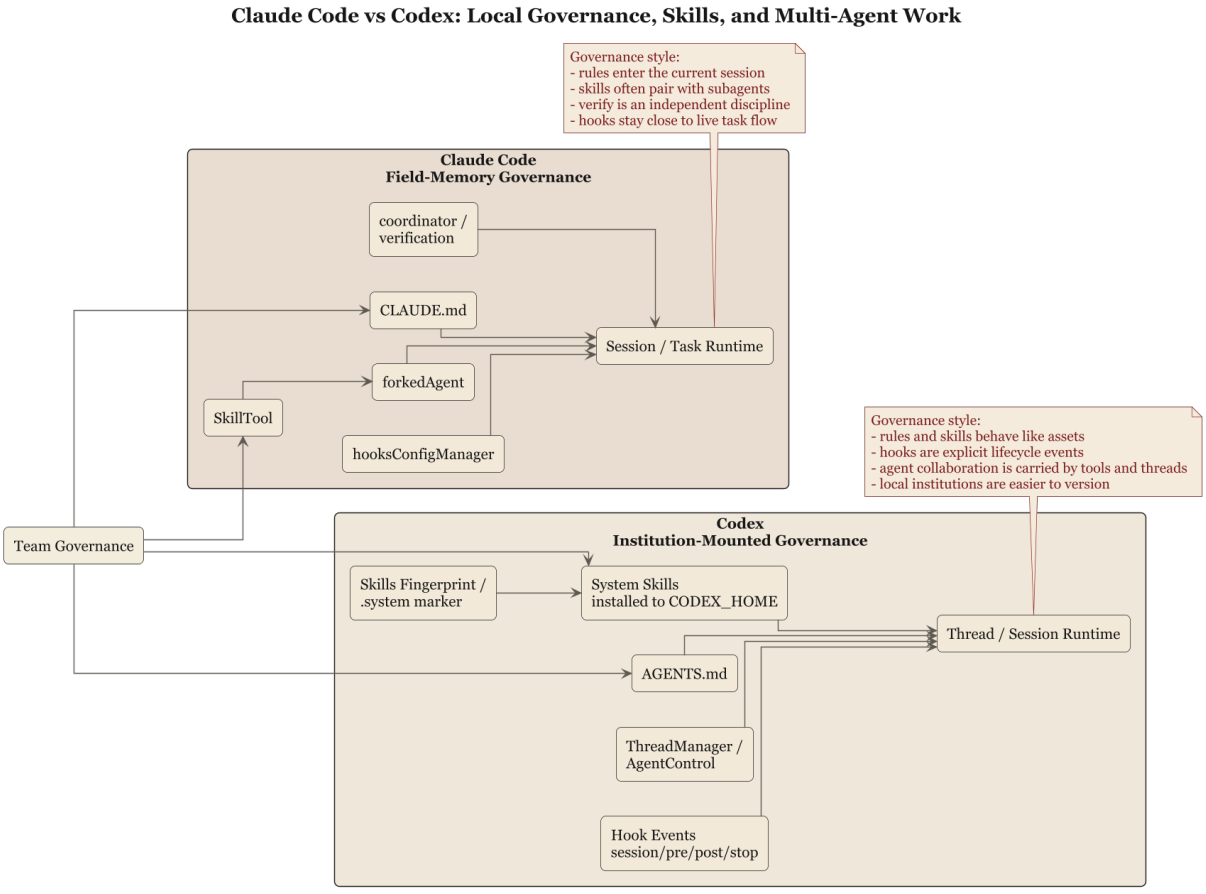

Figure 5: Local Governance Comparison

5.1 Any deployable agent becomes local

Any general-purpose coding agent that starts doing real work for a team eventually collides with the same fact: companies have their own rules, repositories have their own rules, directories have their own rules, and people have their own peculiar habits. If a system cannot absorb those local institutions, it remains trapped in demo environments.

Both Claude Code and Codex offer answers. They simply point in different directions.

5.2 Claude Code: local institutions as field memory

Much of Claude Code's localizing capacity lands in:

- `CLAUDE.md`
- skills
- hooks
- session memory

Together these feel strongly like accumulated field experience:

- `CLAUDE.md` tells the system what counts as common sense in this repo, this directory, and this team
- skills package repeatable workflows
- hooks attach team governance to lifecycle points
- session memory prevents each turn from starting human life over from zero

What all four share is proximity to the task scene. The point is to get rules into the current session and into current execution rather than to establish one eternal organization-wide constitution first. Claude Code resembles an engineer willing to carry a notebook and copy down local custom wherever it goes.

That is highly practical. It fits environments where multiple projects, directories, and local constraints coexist. The downside is obvious too: without deliberate cleanup, knowledge expands as field patches.

5.3 Codex: local institutions as structured injection and event systems

Codex also has skills, local rules, and hooks, but the temperament is more institutional.

Start with skills. `skills/src/lib.rs` shows that system skills are installed into `CODEX_HOME/skills/.system` and tracked with hashes or fingerprints. That is revealing, because it means a skill in Codex is not merely text temporarily read into context. It is an installed, managed, versionable asset.

What matters even more is that Codex also decides when those assets need to be refreshed. `install_system_skills()` computes a fingerprint for the embedded skills and skips reinstalling them when the marker already matches. Only mismatches trigger removal and rewrite of the cached system-skills directory. That small detail is a strong signal that Codex treats skills like deployable assets rather than like templates casually reread at startup.

Then look at `AGENTS.md`. In Codex this does not merely mean "read a local note." It comes with ideas of scope and hierarchy. In other words, local rules are not only content. They carry positional relationships.

Finally, look at hooks. `hooks/src/engine/mod.rs` splits hook events explicitly into:

- `session_start`
- `pre_tool_use`
- `post_tool_use`
- `user_prompt_submit`
- `stop`

Each handler also carries structured information such as event name, matcher, timeout, status message, source path, and display order. This makes Codex hooks feel less like "drop in a callback wherever convenient" and more like a formal lifecycle event system.

And the engine goes one step further: it separates `preview_*` paths from `run_*` paths, which means the system can first explain which handlers would fire before actually executing them. On Windows it even disables `codex_hooks` with an explicit warning because support is not complete. In other words, Codex tries to make hook capability itself explainable.

5.4 Claude Code absorbs experience; Codex mounts institutions

Viewed side by side, the difference becomes very clear.

Claude Code's local governance is closer to continuously absorbing field experience into the neighborhood of the main loop. It is strong at making the agent learn quickly how things are done here.

Codex's local governance is closer to mounting local rules onto an explicit control plane and lifecycle system. It is strong at making rules not only readable but also categorized, ordered, installed, and triggered.

That produces two different team feels.

Claude Code's team feel is like an old employee who knows the floor and can read the room.

Codex's team feel is like a newcomer with strong institutional instincts: first post the rules, then start coordinating the work.

5.5 Different consequences for organizational reproducibility

This difference matters most when an organization tries to reproduce itself.

If a system relies mainly on injected field experience, it adapts faster to a new repository and remains effective more easily in dense local context. But when it spreads to many teams, it usually requires extra editorial work, otherwise everyone writes their own `CLAUDE.md`, invents their own skills, and the organization starts printing its own textbooks by province.

If a system relies mainly on structured injection and lifecycle mounting, it has greater expansion potential. Rules are easier to distribute consistently, version, and audit. The price is learning cost: the team must first accept more explicit institution.

It is a classic engineering tradeoff:

- the closer you stay to the scene, the more elastic the system becomes

- the more institutionalized you become, the easier you are to reproduce

Neither path guarantees happiness. The real determinant is which kind of stability the team actually needs.

5.6 Chapter conclusion

This chapter can be summarized as:

> Claude Code tends to turn local governance into field memory and runtime injection, while Codex tends to turn local governance into structured assets and lifecycle event systems.

That is not just another way of saying "both support skills and hooks."

The difference is that Claude Code asks, "How do we make the agent work here more like a local?" while Codex asks, "How do we make local rules enter a governable institutional framework?"

The next chapter moves into a higher-risk layer: multi-agent work, verification, persistent state, and recovery. Once several agents begin working at once, rules alone are not enough; responsibility has to be partitioned too.

Chapter 6 Delegation, Verification, and Persistent State: Who Stops the System from Grading Itself

6.1 The real problem in multi-agent systems is responsibility

When people hear "multi-agent," they often react as if they were hearing a corporate headcount plan and immediately imagine higher efficiency. But the truly difficult part is never the mere presence of more agents. It is how responsibility gets divided.

If the same system executes, summarizes, verifies, and casually writes its own review, the result is usually comforting and not especially reliable: good job.

Claude Code is unusually clear-eyed about this. As the earlier material already showed, it separates explore, execute, synthesis, and verification, and it treats verify as an independent discipline rather than as a polite closing flourish. That matters because it means the system refuses to let "done" be declared solely by the executing agent.

Codex is clearly moving down the same road. The many agent-related tools in `tools/src/lib.rs`, such as `create_spawn_agent_tool_v*`, `create_wait_agent_tool_v*`, `create_send_message_tool`, and `create_close_agent_tool_` show that delegation in Codex is formal tool capability rather than runtime black magic.

6.2 Claude Code: multi-agent work serves runtime separation of responsibility

Claude Code's multi-agent mechanism remains centered on the main loop and task progression. It is close to saying: the primary agent should not do everything itself, and it definitely should not both implement and certify the implementation.

So multi-agent work is primarily used to handle:

- outsourced exploration tasks
- split implementation tasks
- synthesis of results
- independent verification

This architecture fits its overall temperament. Claude Code's strength already lies in runtime orchestration, so multi-agent naturally becomes part of the governance framework for getting the current task forward. It did not begin with a grand agent platform and then insert tasks into it. It began with field-dispatch problems and developed agent partitioning from there.

6.3 Codex: multi-agent work serves explicit tool-mediated collaboration

In Codex, delegation is more visibly defined as tool interface. That pushes multi-agent closer to the status of a formal subsystem.

That has two direct effects.

First, delegation actions become easier to record, audit, and compose, because they are explicit tool calls rather than hidden runtime maneuvers.

Second, collaboration aligns more naturally with threads, state, and approval systems. Since Codex already treats thread, rollout, and policy as first-class infrastructure, multi-agent work slips into that same infrastructure rather than remaining a local field technique.

That formality is not abstract. In `agent_tool.rs`, `spawn_agent`, `send_input`, `wait_agent`, and `close_agent` each have their own schemas; `send_input`

distinguishes `interrupt=true` from default queued delivery; `wait_agent` is parameterized with default, minimum, and maximum timeout ranges; and `close_agent` explicitly documents closing open descendants as well. So Codex is not merely offering "ask another agent for help." It is turning preemption, waiting, and cleanup into protocol fields.

This is a strong foundation for treating multi-agent behavior as platform capability. It may not always feel as nimble, but it is easier to maintain durably.

6.4 Persistent state keeps verification from becoming etiquette

Verification so often turns ceremonial for one major reason: the system lacks a good enough state handoff. What was just done, why it was done, which tools were used, and which files were touched all matter. If that information lives only in the executing agent's head, the verification phase turns into a performance that looks serious but is starved of material.

Claude Code addresses this by keeping session state, tool results, and recovery branches continuously visible inside runtime, then pairing that continuity with an independent verification discipline to reduce self-flattery.

Codex is more likely to provide explicit material foundations for verification through thread, rollout, message history, and state DB bridge structures. A system with session-archive awareness is simply better at answering, "What exactly happened just now?"

That means the two systems are not in conflict on verification. They repair different deficits:

- Claude Code repairs the problem of an executor becoming too immersed in the live scene
- Codex repairs the problem that collaboration must leave structured evidence

6.5 Different attitudes toward recovery and closure

Multi-agent systems face another practical question: how do they close things out?

A great many details in Claude Code show that it cares deeply about task cleanup, parent-child abort propagation, subagent lifecycle hooks, and similar concerns. In its world, multi-agent work is first a live runtime phenomenon, and if the live scene goes wrong, the system must be able to close it down quickly.

Codex, judging by toolized agents and thread-state structures, leans more toward bringing agent lifecycle under explicit state management and invocation protocol. It cares not only whether a subagent died, but also how that delegated act should persist as a system event.

Again, the difference is temperamental:

- Claude Code resembles a chief engineer on site, worrying about what holes remain after people leave
- Codex resembles an organizer with project infrastructure, worrying whether every collaboration act entered the record system

6.6 Chapter conclusion

The conclusion is not difficult to state:

> Claude Code's multi-agent design emphasizes runtime separation of responsibility and field closure, while Codex's multi-agent design emphasizes tool-mediated delegation, state handoff, and auditable collaboration.

Both are trying to stop the system from giving itself inflated grades.

Claude Code leans more on role separation and verification discipline.

Codex leans more on explicit interfaces, thread state, and collaboration records.

The final chapter compresses the previous six into one overall judgment and answers the book's title question directly: convergence through different roads, or genuinely different species?

Chapter 7 Convergence and Divergence

7.1 Start with the part where they converge

If I had to give the shortest conclusion first, I would say: yes, they genuinely converge.

The reason is straightforward. Neither Claude Code nor Codex treats the model as an executor worthy of direct trust. Both systems accept:

- prompt does not control everything
- tools must be constrained
- long sessions require state governance
- local rules must enter the system
- multi-agent execution requires role partitioning and verification

In other words, both have already moved beyond the naive stage where people imagine that stronger models will somehow dissolve systems problems by themselves. Once a system reaches this point, it no longer treats an agent as merely a chatbot with a few tools attached.

So at the level of direction, they do meet at the same destination: harness is the true control layer, and the model is the most productive but also most unstable component underneath it.

7.2 Now the part where they branch

But it would be far too rough to call them fundamentally identical.

Claude Code’s main axis looks something like this:

- begin from the query loop
- handle continuity in runtime
- preserve order with compaction, tool orchestration, interrupts, and recovery
- connect field rules and team institutions through skills, hooks, and verification

Codex’ s main axis looks more like this:

- begin from explicit module boundaries and explicit control layers
- turn instructions into fragments
- turn tools into schemas
- turn execution boundaries into policy
- turn sessions into thread / rollout / state
- turn local rules and hooks into structured assets and event systems

The first feels like a system grown from mechanical experience.

The second feels like a system grown from institutional design.

That is where the branching really lies. The difference is not destination. It is skeleton.

7.3 If I had to give them a harsher but more accurate label

I would even be willing to call them two different political forms of harness.

Claude Code is closer to a runtime republic. A great deal of power is concentrated in the main loop and field dispatch, and order is maintained through continuous negotiation with reality. It is not anti-institution; institutions just tend to exist in service of the live session.

Codex is closer to a constitutional control plane. Power is first written into types, fragments, policy, threads, and event systems. Runtime still judges, of course, but it judges inside a more explicit framework.

That is an exaggerated description, but it clarifies an important point: harness is never only a pile of technical parts. It is also a way of distributing power. Who

defines the boundary, who interprets state, and who owns the final authority of execution all eventually appear in architecture.

7.4 What later builders should learn

If a team intends to build its own coding agent, the true value of this comparison is not side-picking. It is avoiding two categories of mistake.

The first mistake is thinking a feature table is enough. It is not. A team has to decide what its primary contradiction is. Is the real problem that long sessions go out of control, or that rule sources are too scattered and permission boundaries remain unclear? Different contradictions push you toward different harness shapes.

The second mistake is trying to splice together the most attractive features of both systems without making actual tradeoffs. In engineering, what is most dangerous is often not tradeoff itself but the refusal to make one. If you want fully dynamic runtime flexibility and a completely explicit structured control plane at the same time, you usually end up with neither.

The saner approach is:

- if you fear loss of control on site, strengthen the runtime heartbeat first
- if you fear institutional drift, make instruction, tool, policy, and state explicit first
- once the primary contradiction stabilizes, gradually build the opposite side

One more caution belongs here. In practice many younger systems do neither job fully. They neither harden runtime discipline nor make the control layer truly explicit. Instead they take a third, easier-looking road: keep stuffing more bootstrap files, role descriptions, skill explanations, and workspace text into prompt, hoping that informational fullness will compensate for a weak skeleton. Such systems often appear workable in the short term, but over longer sessions they expose the same double failure: tokens are expensive, and working semantics are still unstable.

7.5 Final judgment

The question in the title can now be answered directly.

They are convergence through different roads.

They are also distinct branches of the same larger problem.

“Convergence” names the reality they both accept: the model is unreliable, and harness is where order comes from.

“Different branches” names the different political economy through which that reality is implemented: Claude Code trusts runtime discipline more, while Codex trusts explicit control layers more.

Systems that rely mainly on prompt stacking to hold context together sit in a less settled middle zone. They have already realized that models forget and drift, so they add memory, skills, and compaction. But if context governance still follows a “inject first, rescue later” logic, then the system still has not decided clearly which layer should own order.

Neither path is inherently nobler. The real question is this: in which layer is your system prepared to cage uncertainty?

The location of that cage determines what the system will later become.

Chapter 8 If You Need to Build Your Own Harness: Whom to Learn From, and What to Learn First

8.1 The real use of comparison is to avoid unnecessary detours

The least interesting ending for a comparison book is to push the reader back into a consumer posture: choose A or B. Engineering systems are not headphones. You do not buy them through ranking charts. The useful question is this: if you are about to build your own harness, or refactor the agent you already have, whose lessons should you learn first, and which part should you learn first?

Claude Code and Codex offer two opening moves, not two final answers.

Claude Code is useful because it warns you not to make runtime problems sound too elegant. What actually drags systems down are often the dirty jobs inside the query loop: closing tool-result ledgers, managing context inflation, recovering from interrupts, cleaning up subagents, keeping verification independent, and preventing failure handling from becoming a loop. Anyone who treats those problems lightly usually ends up with a system that looks smart and feels lethal to operate.

Codex is useful because it warns you not to let the control layer dissolve into tacit understanding. Instruction sources, tool schemas, approval policy, thread state, hook events, and skill assets become easier to govern the earlier they are made explicit. Teams that keep asking runtime improvisation to replace institution eventually discover that the whole system resembles a shed

built out of verbal agreement.

8.2 Three common team shapes, three opening directions

Type one: the team already has an agent prototype, but long sessions frequently lose control

This kind of team usually needs Claude Code first.

Their problem is rarely that "the control plane is underdefined." Their problem is that the system cannot stay alive long enough. Typical symptoms include:

- context gets noisier over time
- tool-call chains break
- state becomes unclear after interruption
- no one closes subagent work cleanly
- verification degenerates into something people merely say

When those are your symptoms, the first thing to repair is runtime heartbeat, not more configuration. Stabilize the loop first. Institutional aesthetics can wait.

Type two: the team already has many rules, but rule sources are scattered and permission boundaries are unclear

This kind of team usually needs Codex first.

Their problem is not that the live scene is impossible to survive. Their problem is that the system becomes harder and harder to govern. Common symptoms include:

- local rules are scattered everywhere
- no one can say which constraints live in prompt and which live in tools
- approval logic is mixed into code and hard to explain
- once multiple extensions are introduced, boundaries blur further

What these teams need is an explicit control layer. Turn instruction, tool, policy, and thread into explicit concepts before asking runtime to work inside them.

Type three: there is no mature system yet and the team is starting from scratch

This is the most dangerous case, because it is easiest to envy the strengths of both sides at once and build a failed compromise.

A steadier route is usually:

- choose one primary contradiction
- design the main skeleton around that contradiction
- add the opposite side only at minimum level for now

If your first-phase risk is mainly "the model will behave recklessly," start with runtime discipline in the Claude Code sense.

If your first-phase risk is mainly "the team will lose institutional order," start with an explicit control layer in the Codex sense.

The worst move is trying to learn both sides fully at the same time and ending up with neither a stable main loop nor a clear control plane.

8.3 What to learn from Claude Code, and what to learn from Codex

This is worth stating more bluntly.

Prioritize learning from Claude Code when it comes to:

- the state mind-set of the query loop
- compaction and context governance
- tool orchestration and interrupt handling
- subagent lifecycle and verification independence
- treating failure paths as main paths

Prioritize learning from Codex when it comes to:

- instruction fragmentation
- tool schemas
- explicit expression of approval and policy
- infrastructure for thread / rollout / state
- hook events and skill-asset management

This is not compromise for its own sake. It contains a hidden requirement: you must know what you are learning and why. The correct reason to borrow something is that it repairs your weakness, not merely that someone else already built it.

If you carry the context-governance judgments from the first volume into third-party harnesses, you can identify a common but costly route much more easily. It does not begin by separating context into units with different lifetimes, duties, and entry costs. Instead it tries to pack bootstrap files, skill descriptions, identity settings, and workspace text into prompt as far as possible, then leans on truncation, compaction, and recovery chains once the window gets tight.

Systems on this route may still have memory, skills, and compaction, and they may even impose upper bounds. But the governing axis remains "inject first, rescue later." That is the real problem. Once context is organized mainly by piling up text, token waste is only the first cost. The deeper cost is signal dilution: the model sees more things, but is not necessarily clearer about which working semantics matter for the next action.

If you wanted a one-sentence summary of the three routes, it would look like this:

Claude Code treats context more like working memory, first deciding what must survive and what should be compressed.

Codex treats context more like structured units, first deciding source type, scope, and state handoff.

Systems in the OpenClaw family treat context more like a prompt container, first deciding what else can still be packed in before the limit.

That is why teams often begin by feeling that these systems are "more informed," then later complain about two things at once: tokens are burned quickly, and the quality does not become steadily better as context gets fatter.

The system is solving how much can be inserted, not what must be preserved for continued work.

8.4 One dangerous misconception: explicitness and flexibility are not natural enemies

System builders often rely on a lazy false opposition.

Say "explicit control layer" and they imagine a system that must become heavy, slow, and rigid.

Say "runtime flexibility" and they imagine experience can carry the system now while structure is deferred to later.

Neither instinct is very intelligent. Explicitness is not inherently rigid, and flexibility is not inherently chaotic. The real question is whether you have defined clearly which things must be explicit and which things can be left to field judgment.

Claude Code's strength is not that it rejects structure. Its strength is that it knows which troubles must be faced directly inside runtime.

Codex's strength is not that it rejects flexibility. Its strength is that it knows which boundaries will turn into endless disputes if they are not declared early.

A good third harness should not average the two. It should distinguish:

- which rules must be written down first
- which judgments can remain in runtime
- which state must persist
- which experience only needs to be retained temporarily inside session memory

8.5 A practical order of operations for later builders

If I were starting a harness from zero, I would recommend this sequence:

1. define high-risk actions and the minimum permission model
2. define the main loop or thread lifecycle

3. define context governance and recovery paths
4. define skills, local rules, and hooks
5. only then scale into multi-agent, platform capability, and a more complex ecosystem

This sequence is not glamorous, but it roughly follows the order in which incidents appear. In engineering, many design orders should be arranged according to the order of failure, not according to what looks nicest in a demo.

8.6 Chapter conclusion

This chapter only wants to leave one plain sentence behind:

> Learn from Claude Code mainly to understand how a system stays stable on site; learn from Codex mainly to understand how a system sustains order over time inside an organization.

Teams that learn only the first tend to become rich in experience and poor in institution.

Teams that learn only the second tend to produce elegant institutions and fragile field behavior.

The better move is not side-picking. It is deciding which bone your system must grow first according to its primary contradiction.

Appendix A Source Map: Which Files This Comparison Primarily Relies On

This appendix does one thing only: it states which files each chapter's judgments mainly rely on. It is not a directory of reproduced source, nor does it imply that the book ships source code together with its arguments.

The same boundary applies here:

- only the minimum engineering citation and module location is used
- the body of Claude Code or Codex implementation is not reproduced
- no long source transcription is provided

A.1 Primary references on the Claude Code side

Overall control plane and prompt:

- `src/constants/prompts.ts`
- `src/utils/systemPrompt.ts`
- `src/utils/claudemd.ts`
- `src/memdir/memdir.ts`

Runtime loop and recovery:

- `src/query.ts`
- `src/QueryEngine.ts`
- `src/services/compact/autoCompact.ts`

- `src/services/compact/compact.ts`

Tools and permissions:

- `src/services/tools/toolOrchestration.ts`
- `src/services/tools/toolExecution.ts`
- `src/services/tools/StreamingToolExecutor.ts`
- `src/hooks/useCanUseTool.tsx`
- `src/tools/BashTool/prompt.ts`
- `src/tools/BashTool/bashPermissions.ts`

Multi-agent work, skills, and hooks:

- `src/utils/forkedAgent.ts`
- `src/coordinator/coordinatorMode.ts`
- `src/tasks/LocalAgentTask/LocalAgentTask.tsx`
- `src/tools/SkillTool/SkillTool.ts`
- `src/tools/SkillTool/prompt.ts`
- `src/utils/hooks/hooksConfigManager.ts`

A.2 Primary references on the Codex side

Core module skeleton:

- `core/src/lib.rs`
- `tools/src/lib.rs`
- `skills/src/lib.rs`
- `hooks/src/lib.rs`

Instruction fragments and user injection:

- `instructions/src/lib.rs`
- `instructions/src/fragment.rs`
- `instructions/src/user_instructions.rs`
- `docs/agents_md.md`

Tools, approvals, and execution policy:

- `tools/src/local_tool.rs`
- `tools/src/agent_tool.rs`
- `execpolicy/src/lib.rs`
- `docs/execpolicy.md`
- `docs/sandbox.md`

Threads and state:

- `sdk/typescript/src/thread.ts`
- `core/src/lib.rs`
- `core/src/thread_manager.rs`
- `core/src/rollout.rs`
- `core/src/state_db_bridge.rs`
- `core/src/message_history.rs`

Hook event engine:

- `hooks/src/engine/mod.rs`

A.3 Chapter-to-file mapping

Chapter 1:

- Claude Code's `query.ts`, `toolOrchestration.ts`
- Codex's `core/src/lib.rs`

Chapter 2:

- Claude Code's prompt assembly and `CLAUDE.md`
- Codex's `fragment.rs`, `user_instructions.rs`

Chapter 3:

- Claude Code's query loop and `QueryEngine`
- Codex's `thread.ts`, plus exposed modules around `thread_manager`, `rollout`, and `state_db_bridge`

Chapter 4:

- Claude Code’s tool orchestration, Bash restrictions, and permission semantics
- Codex’s `tools/src/lib.rs`, `local_tool.rs`, and `execpolicy/src/lib.rs`

Chapter 5:

- Claude Code’s skill, hook, and memory system
- Codex’s skills system and hook engine

Chapter 6:

- Claude Code’s forked-agent design and verification discipline
- Codex’s agent-tool set and its thread / rollout / state structures

Chapter 7:

- the cumulative judgment produced by the whole file set above

Appendix B Checklists: How to Tell Whether Your Harness Resembles Claude Code, Codex, or an Unfinished Prototype

If comparison cannot be reduced into checklists, what remains is usually a set of well-phrased but hard-to-use conclusions. This appendix compresses the previous chapters into questions a team can actually discuss.

B.1 Control-plane checklist

Check these questions:

- Are local rules assembled as free text, or injected as typed fragments with bounded structure?
- Can the sources, scope, and precedence of instructions be explained clearly?
- Which parts of prompt are real control plane, and which are merely output style?
- Are team rules closer to `CLAUDE.md`-style field notes, or closer to `AGENTS.md`-style structured institutional entry points?

If these questions cannot be answered clearly, the control plane is probably still at the "good enough to use" stage rather than the "good enough to govern" stage.

B.2 Continuity checklist

Check these questions:

- Is continuity maintained mainly by a main loop, or mainly by thread / rollout / state?
- After interruption, who guarantees closure for tool ledger, message sequence, and state handoff?
- When long sessions inflate, is there an explicit compact / truncation / recovery mechanism?
- Are thread IDs, session indexing, and persisted state first-class concepts?

If long sessions rely mostly on the model to "remember," there is usually no need to continue the evaluation.

B.3 Tool and approval checklist

Check these questions:

- Are tools runtime objects, or schema-defined interfaces?
- Are approvals based mainly on field judgment, or on an explicit policy / rule system?
- Are dangerous tools specially governed, rather than treated like ordinary read operations?
- Can boundaries such as `workdir`, network, sandbox, and approval be expressed explicitly?

If the system can only answer "we also have permission controls," that usually means the permission system has not actually been designed.

B.4 Local governance checklist

Check these questions:

- Can local rules be layered by directory, team, and task type?

- Can skills be treated as reusable slices of institution rather than merely long prompts?
- Are hooks attached to explicit lifecycle events?
- Do skills, rules, and hooks have version, origin, and trigger boundaries?

When team governance relies mainly on oral explanation, the system eventually learns to pretend it understands.

B.5 Multi-agent and verification checklist

Check these questions:

- Is multi-agent used for parallelism, or for separation of responsibility?
- Is there an independent verification mechanism?
- Is delegation represented as explicit tools or explicit state events?
- When a child agent fails, times out, or is cancelled, who owns cleanup?

A multi-agent system that cannot verify independently and cannot close out cleanly is usually just parallelized disorder.

B.6 Which kind of system are you closer to?

Signals that you are closer to Claude Code:

- you care most about query loop, tool orchestration, interrupts, compaction, and recovery
- you are good at getting rules into the live session quickly
- you care primarily about how an agent keeps running inside complex tasks

Signals that you are closer to Codex:

- you care most about instruction fragments, tool schemas, approval policy, thread / rollout / state
- you are good at turning local rules into structured assets

- you care primarily about how an agent is governed durably inside an organization

Signals that you are closer to an unfinished prototype:

- you can recite vocabulary from both sides, but cannot explain who owns order
- you have many capability entry points, but no clear recovery path
- you have many rule texts, but no scope or precedence
- you have multi-agent execution, but no separation of responsibility and no closure mechanism

B.7 Six final questions

If time is short, ask only these six:

- Who owns the final control, the model or the harness?
- Does continuity live mainly in the loop, or in threads and state?
- Before tools act, who stops the last dangerous move?
- How do local rules enter the system, and how are they layered?
- Who owns verification, and how is it kept independent?
- After something goes wrong, what evidence lets the team trace the path back?

Once these six questions are asked, the system's political family usually reveals itself.